



第二次作业

并行程序设计 with 算法（理论）

元朗曦

23336294

题 1.

1. OpenMP 主要适用于哪一类内存系统？`#pragma omp parallel` 的作用是什么？`num_threads(n)` 子句的作用是什么？它是否一定保证实际启动 n 个线程？
2. 在 OpenMP 并行区域中，什么是 **shared** 变量，什么是 **private** 变量？如果一个变量在并行区域之前已经声明，且没有显式指定作用域，它默认通常属于哪一种？

解.

1. OpenMP 主要适用于 **shared-memory system**（共享内存系统），因为多个线程可以访问同一地址空间中的数据。

`#pragma omp parallel` 用于创建一个并行区域：程序执行到该指令时，会形成一个线程组，由多个线程共同执行该区域中的代码。

`num_threads(n)` 用于请求该并行区域使用 n 个线程，但它不一定保证实际启动恰好 n 个线程；运行时系统仍可能受动态线程调整、系统资源或实现限制影响，最终线程数可能不同。

2. **shared** 变量表示并行区域内多个线程访问的是同一份存储，某线程对它的修改对其他线程可见；**private** 变量表示每个线程各自拥有一份独立副本，线程之间互不共享。

若变量在并行区域之前已经声明，且没有显式指定数据作用域，它通常默认是 **shared** 变量。

题 2. 阅读以下代码片段：

```
int a[12] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12};
int sum = 0;

#pragma omp parallel for num_threads(3) schedule(static, 1) reduction(+:sum)
for (int i = 0; i < 12; i++) {
    sum += a[i];
}
```

假设程序实际启动了 3 个线程。`schedule(static, 1)` 表示每次分配 1 次循环迭代，并按线程编号轮询分配给各线程。

1. 分别写出线程 0、线程 1、线程 2 负责的循环迭代编号 i 。
2. 分别计算 3 个线程对应的局部和，并写出最终 `sum` 的值。
3. 如果去掉 `reduction(+:sum)`，多个线程直接执行 `sum += a[i]`，是否可能产生竞态条件？请回答“是”或“否”，并简要说明原因。

解.

1. 按照 `schedule(static, 1)` 的轮询分配方式：

- 线程 0: $i = 0, 3, 6, 9$;

- 线程 1: $i = 1, 4, 7, 10$;
- 线程 2: $i = 2, 5, 8, 11$ 。

2. 各线程的局部和分别为:

- 线程 0: $a[0] + a[3] + a[6] + a[9] = 1 + 4 + 7 + 10 = 22$;
- 线程 1: $a[1] + a[4] + a[7] + a[10] = 2 + 5 + 8 + 11 = 26$;
- 线程 2: $a[2] + a[5] + a[8] + a[11] = 3 + 6 + 9 + 12 = 30$ 。

因此最终:

$$\sum = 22 + 26 + 30 = 78.$$

3. 是。因为 `sum += a[i]` 不是一个不可分割的原子操作, 而是包含读取、计算、写回多个步骤; 多个线程同时更新同一个共享变量 `sum` 时, 可能发生写入覆盖, 从而造成竞态条件。

题 3.

1. 在 MPI 程序中, `MPI_Init` 和 `MPI_Finalize` 分别有什么作用? 为什么通常要求 MPI 调用位于二者之间?
2. 什么是 **rank**? 如果 `MPI_COMM_WORLD` 中共有 p 个进程, 这些进程的 rank 编号范围是什么? `MPI_Comm_size` 和 `MPI_Comm_rank` 分别用于获得什么信息?
3. 在点对点通信中, `MPI_Send` 与 `MPI_Recv` 的匹配主要依赖哪些信息? 如果接收操作指定的 `source` 或 `tag` 与实际发送消息不匹配, 会发生什么?

解.

1. `MPI_Init` 用于初始化 MPI 运行环境, 使进程能够开始使用 MPI; `MPI_Finalize` 用于结束 MPI 环境并完成相关清理工作。

通常要求 MPI 调用位于二者之间, 是因为在 `MPI_Init` 之前 MPI 环境尚未建立, 而在 `MPI_Finalize` 之后 MPI 环境已经结束, 继续调用 MPI 函数通常是不合法的。

2. **rank** 是通信域中每个进程的唯一编号。若 `MPI_COMM_WORLD` 中共有 p 个进程, 则 rank 的范围是 $0, 1, \dots, p-1$ 。

`MPI_Comm_size` 用于获得某个通信域中的进程总数; `MPI_Comm_rank` 用于获得当前进程在该通信域中的 rank 编号。

3. 点对点通信的匹配主要依赖通信域 (communicator)、源进程 (source)、目标进程 (destination) 以及消息标记 (tag); 发送与接收双方还应在数据类型、数量等方面保持兼容。

如果接收操作指定的 `source` 或 `tag` 与实际发送消息不匹配, 则该接收操作不会匹配到这条消息; 若没有其他满足条件的消息到达, 它通常会一直阻塞等待。

题 4. 使用梯形积分方法近似计算函数 $f(x) = x$ 在区间 $[0, 1]$ 上的积分。设总梯形数 $n = 4$, 线程数 `thread_count = 2`, 采用连续块划分, 每个线程负责相同数量的连续梯形。

梯形积分公式为：

$$h = \frac{b-a}{n},$$
$$\approx h \left[\frac{f(x_0)}{2} + f(x_1) + f(x_2) + \dots + f(x_{n-1}) + \frac{f(x_n)}{2} \right].$$

1. 计算步长 h ，以及每个线程负责的梯形数 `local_n`。
2. 分别写出线程 0 和线程 1 负责的积分子区间 `[local_a, local_b]`。
3. 分别计算两个线程的局部积分结果 `my_result`，并求最终的 `global_result`。

解.

1. 步长为：

$$h = \frac{1-0}{4} = \frac{1}{4} = 0.25.$$

每个线程负责的梯形数为：

$$\text{local_n} = \frac{n}{\text{thread_count}} = \frac{4}{2} = 2.$$

2. 每个线程负责 2 个连续梯形，因此：

- 线程 0 负责区间 $[0, 0.5]$ ；
- 线程 1 负责区间 $[0.5, 1]$ 。

3. 线程 0 的局部积分为：

$$\text{my_result}_0 = 0.25 \left[\frac{f(0)}{2} + f(0.25) + \frac{f(0.5)}{2} \right] = 0.25[0 + 0.25 + 0.25] = 0.125.$$

线程 1 的局部积分为：

$$\text{my_result}_1 = 0.25 \left[\frac{f(0.5)}{2} + f(0.75) + \frac{f(1)}{2} \right] = 0.25[0.25 + 0.75 + 0.5] = 0.375.$$

因此最终结果为：

$$\text{global_result} = 0.125 + 0.375 = 0.5.$$

题 5. 某程序需要对数组元素进行求和。为了让循环迭代更均匀地分配到不同线程，程序使用了 `schedule(static, 1)`，即每次分配 1 次循环迭代，并按线程编号循环分配给各线程。

```
int a[8] = {1, 2, 3, 4, 5, 6, 7, 8};
int sum = 0;

#pragma omp parallel for num_threads(4) schedule(static, 1) reduction(+:sum)
for (int i = 0; i < 8; i++) {
```

```
    sum += a[i];  
}
```

假设程序实际启动了 3 个线程。

1. 根据 `schedule(static, 1)` 的分配方式，分别写出线程 0、线程 1、线程 2 负责的循环迭代编号 i 。
2. 分别计算 3 个线程对应的局部和，并写出最终 `sum` 的值。
3. 如果去掉 `reduction(+:sum)`，多个线程直接执行 `sum += a[i]`，是否可能产生竞态条件？请回答“是”或“否”，并简要说明原因。

解.

1. 题目说明程序实际启动了 3 个线程，因此按这 3 个线程轮询分配迭代：

- 线程 0: $i = 0, 3, 6$;
- 线程 1: $i = 1, 4, 7$;
- 线程 2: $i = 2, 5$ 。

2. 各线程的局部和分别为：

- 线程 0: $a[0] + a[3] + a[6] = 1 + 4 + 7 = 12$;
- 线程 1: $a[1] + a[4] + a[7] = 2 + 5 + 8 = 15$;
- 线程 2: $a[2] + a[5] = 3 + 6 = 9$ 。

因此最终：

$$\sum = 12 + 15 + 9 = 36.$$

3. 是。去掉 `reduction(+:sum)` 后，多个线程会同时读写共享变量 `sum`；由于 `sum += a[i]` 包含读取、加法和写回多个步骤，不同线程的更新可能彼此覆盖，因此会产生竞态条件。